

```

print("KEERTESH U 1BM24CS132")
board = ["1", "2", "3",
         "4", "5", "6",
         "7", "8", "9"]

players = {
    "X": "Player 1",
    "O": "Player 2"
}

winning_combinations = (
    (0, 1, 2),
    (3, 4, 5),
    (6, 7, 8),
    (0, 3, 6),
    (1, 4, 7),
    (2, 5, 8),
    (0, 4, 8),
    (2, 4, 6)
)

occupied = set()

def display_board():
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("--+---")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---")
    print(board[6], "|", board[7], "|", board[8])
    print()

def check_winner(symbol):
    for combination in winning_combinations:
        if all(board[position] == symbol for position in combination):
            return True
    return False

def make_move(symbol):
    while True:
        choice = input(
            f"{players[symbol]} ({symbol}), choose a position (1-9): "
        )

        if not choice.isdigit():
            print("Please enter a valid number.")
            continue

        position = int(choice) - 1

        if position < 0 or position > 8:
            print("Please choose a position between 1 and 9.")
            continue

        if position in occupied:
            print("This position is already occupied.")
            continue

        board[position] = symbol
        occupied.add(position)
        break

print("Welcome to Tic-Tac-Toe!")

current_player = "X"

while True:
    display_board()

    make_move(current_player)

    if check_winner(current_player):
        display_board()
        print(players[current_player], "wins!")
        break

```

```

    if len(occupied) == 9:
        display_board()
        print("It's a draw!")
        break

    current_player = "O" if current_player == "X" else "X"

print("\nThanks for playing!")

```

KEERTESH U 1BM24CS132
Welcome to Tic-Tac-Toe!

```

1 | 2 | 3
--+-+-
4 | 5 | 6
--+-+-
7 | 8 | 9

```

Player 1 (X), choose a position (1-9): 2

```

1 | X | 3
--+-+-
4 | 5 | 6
--+-+-
7 | 8 | 9

```

Player 2 (O), choose a position (1-9): 1

```

0 | X | 3
--+-+-
4 | 5 | 6
--+-+-
7 | 8 | 9

```

Player 1 (X), choose a position (1-9): 4

```

0 | X | 3
--+-+-
X | 5 | 6
--+-+-
7 | 8 | 9

```

Player 2 (O), choose a position (1-9): 6

```

0 | X | 3
--+-+-
X | 5 | 0
--+-+-
7 | 8 | 9

```

Player 1 (X), choose a position (1-9): 9

```

0 | X | 3
--+-+-
X | 5 | 0
--+-+-
7 | 8 | X

```

Player 2 (O), choose a position (1-9): 8

```

0 | X | 3
--+-+-
X | 5 | 0
--+-+-
7 | 0 | X

```

Player 1 (X), choose a position (1-9): 5

```

print("KEERTESH U 1BM24CS132")
def vacuum_world():
    # 0 indicates CLEAN, 1 indicates DIRTY
    goal_state = {'A': '0', 'B': '0'}
    cost = 0

    print("--- Vacuum Cleaner Environment Setup ---")
    # 1. Get initial vacuum location
    location_input = input("Enter initial location of Vacuum (A/B): ").upper().strip()

    # 2. Get the status of the rooms
    status_input = input(f"Enter status of Room {location_input} (0 for CLEAN, 1 for DIRTY): ").strip()

```

```

other_room = 'B' if location_input == 'A' else 'A'
status_input_complement = input(f"Enter status of Room {other_room} (0 for CLEAN, 1 for DIRTY): ").strip()

# Create current environment dictionary
environment = {location_input: status_input, other_room: status_input_complement}

print(f"\nInitial Environment State: {environment}")
print("Starting simulation...")

# Process Room A
if location_input == 'A':
    # Condition for Room A
    if environment['A'] == '1':
        print("Room A is DIRTY. Sucking the dirt...")
        environment['A'] = '0'
        cost += 1 # Cost for cleaning
        print(f"Room A cleaned. Current Cost: {cost}")
    else:
        print("Room A is already CLEAN.")

    # Move to Room B
    print("Moving to Room B...")
    cost += 1 # Cost for moving

    # Condition for Room B
    if environment['B'] == '1':
        print("Room B is DIRTY. Sucking the dirt...")
        environment['B'] = '0'
        cost += 1 # Cost for cleaning
        print(f"Room B cleaned. Current Cost: {cost}")
    else:
        print("Room B is already CLEAN.")

# Process Room B
else:
    # Condition for Room B
    if environment['B'] == '1':
        print("Room B is DIRTY. Sucking the dirt...")
        environment['B'] = '0'
        cost += 1 # Cost for cleaning
        print(f"Room B cleaned. Current Cost: {cost}")
    else:
        print("Room B is already CLEAN.")

    # Move to Room A
    print("Moving to Room A...")
    cost += 1 # Cost for moving

    # Condition for Room A
    if environment['A'] == '1':
        print("Room A is DIRTY. Sucking the dirt...")
        environment['A'] = '0'
        cost += 1 # Cost for cleaning
        print(f"Room A cleaned. Current Cost: {cost}")
    else:
        print("Room A is already CLEAN.")

# Final Summary
print("\n--- Simulation Results ---")
print(f"Final Environment State: {environment}")
print(f"Goal State Met: {environment == goal_state}")
print(f"Total Path Cost: {cost}")

# Run the program
if __name__ == "__main__":
    vacuum_world()

```

```

KEERTESH U 1BM24CS132
--- Vacuum Cleaner Environment Setup ---
Enter initial location of Vacuum (A/B): B
Enter status of Room B (0 for CLEAN, 1 for DIRTY): 1
Enter status of Room A (0 for CLEAN, 1 for DIRTY): 0

```

```

Initial Environment State: {'B': '1', 'A': '0'}
Starting simulation...
Room B is DIRTY. Sucking the dirt...
Room B cleaned. Current Cost: 1

```

```
Moving to Room A...  
Room A is already CLEAN.
```

```
--- Simulation Results ---  
Final Environment State: {'B': '0', 'A': '0'}  
Goal State Met: True  
Total Path Cost: 2
```